

dyx-blog 前端技术文档

含封面与目录的正式交付版

dyx-blog 项目文档

生成日期：2026-03-30

目录

目录	2
前端实现详解	4
目录	4
1. 项目整体架构	4
1.1 架构总览	4
1.2 技术选型	4
1.3 数据流向	5
2. 模块划分与职责说明	5
2.1 顶层模块	5
2.2 前后台分区方式	5
2.3 布局职责	6
3. 关键依赖与版本清单	6
4. 功能逐点拆解	6
4.1 应用启动与基础装配	6
4.2 前后台路由分区与懒加载	7
4.3 后台登录态与路由守卫	8
4.4 Axios 拦截器与统一错误处理	9
4.5 首页聚合展示	10
4.6 Hero 配置解析	11
4.7 足迹地图渲染	12
4.9 关于页作品区横向滚动优化	13
4.10 WebLayout 主题与导航可见性控制	15
4.9 地图 SDK 延迟加载与地理数据缓存	16
4.10 仪表盘图表与异步组件拆分	17
4.11 后台导航与管理页组织	18
4.12 后台媒体选择与业务裁剪协作	19
5. 数据流与状态管理	20
5.1 组件生命周期	20
5.2 全局状态管理	21
5.1 组件生命周期	21
5.2 全局状态管理	21
5.3 路由守卫	21

5.4 缓存策略	21
6. 安全与性能	22
6.1 性能	22
6.2 安全	22
6.3 SSR/CSR 选型	22
7. 测试与质量保障	23
7.1 测试覆盖现状	23
7.2 自动化流水线	23
7.3 代码规范	23
8. 部署与运维	23
8.1 构建	24
8.2 Nginx 托管与 API 反向代理	24
8.3 环境变量	24
9. 现状结论与补充建议	24
9.1 已具备能力	24
9.2 当前未发现或未实现能力	24
9.3 读者复现建议	25

前端实现详解

目录

- [1. 项目整体架构](#1-项目整体架构)
- [2. 模块划分与职责说明](#2-模块划分与职责说明)
- [3. 关键依赖与版本清单](#3-关键依赖与版本清单)
- [4. 功能逐点拆解](#4-功能逐点拆解)
- [5. 数据流与状态管理](#5-数据流与状态管理)
- [6. 安全与性能](#6-安全与性能)
- [7. 测试与质量保障](#7-测试与质量保障)
- [8. 部署与运维](#8-部署与运维)
- [9. 现状结论与补充建议](#9-现状结论与补充建议)

1. 项目整体架构

1.1 架构总览

前端位于 `frontend/`，是一个基于 Vue 3 + Vite 的单页应用（SPA），前台展示站点与后台管理端共用同一套运行时，通过路由前缀与布局组件进行分区。应用入口在 `frontend/src/main.ts:1-18`，负责装配 Pinia、Vue Router 与 Element Plus。

核心架构图见 `../assets/frontend-architecture.mmd`。

1.2 技术选型

- 框架：Vue 3 (`frontend/package.json:17`)
- 构建工具：Vite 6 (`frontend/package.json:30`)
- 路由：Vue Router 4 (`frontend/package.json:20`)
- 状态管理：Pinia 3 (`frontend/package.json:15`)
- UI 组件：Element Plus (`frontend/package.json:14`)
- 样式体系：Tailwind CSS + SCSS (`frontend/package.json:28`, `frontend/package.json:27`)

- 图表: ECharts + vue-echarts (frontend/package.json:13, frontend/package.json:19)
- 地图: 高德地图 SDK + china-map-data (frontend/src/components/web/HomeFootprintMap.vue:126-127)

1.3 数据流向

典型数据流为:

View ■■ -> API ■■ -> Axios ■■ -> /api ■■ -> Spring Boot ■■
-> ■■■■ -> View ■■

对于后台接口, 请求会额外经过登录态仓库, 自动注入 Bearer Token, 见 frontend/src/api/http.ts:53-59。

2. 模块划分与职责说明

2.1 顶层模块

- src/views/web/: 前台公开页面, 例如首页、博客、关于页、留言页、动态页。
- src/views/admin/: 后台管理页面, 例如仪表盘、媒体管理、文章管理、系统配置。
- src/layouts/: 前后台布局骨架。
- src/router/: 路由定义与守卫。
- src/stores/: 全局状态, 目前主要用于后台认证状态。
- src/api/: 按公共站点与后台管理拆分 API 调用。
- src/utils/: 地图、日期、媒体等工具函数。
- src/styles/: 全局样式与设计 token。

2.2 前后台分区方式

路由定义位于 frontend/src/router/index.ts:9-99:

- / 下挂载 WebLayout, 承载公开站点页面。
- /dyx-manager 下挂载 AdminLayout, 承载后台管理页面。
- /dyx-manager/login 作为独立登录页。

这种设计的直接收益是:

- 部署上只需一套静态资源。
- 共享公共能力（HTTP、样式、构建配置）。
- 后台管理模块仍可用路由元信息进行隔离。

2.3 布局职责

- frontend/src/layouts/WebLayout.vue: 承载前台导航、主题与公开内容容器。
- frontend/src/layouts/AdminLayout.vue:1-235: 提供后台导航分组、头部退出登录按钮、移动端抽屉导航。

AdminLayout.vue:136-177 把后台功能组织为“概览 / 首页管理 / 内容管理 / 个人资料 / 系统”五组，使导航与后端模块划分基本一致。

3. 关键依赖与版本清单

```
| ■■ | ■■ | ■■ |
| --- | --- | --- |
| vue | ^3.5.13 | ■■■■■■ |
| vite | ^6.3.5 | ■■■■■■■■ |
| vue-router | ^4.5.1 | ■■■■ |
| pinia | ^3.0.2 | ■■■■■■■■ |
| element-plus | ^2.10.7 | ■■■■■■■■ |
| echarts | ^6.0.0 | ■■■■■■■■ |
| vue-echarts | ^8.0.1 | Vue ■■■■ |
| tailwindcss | ^3.4.17 | ■■■■■■ |
| sass | ^1.98.0 | SCSS ■■■■ |
| axios | ^1.9.0 | HTTP ■■■■ |
| china-map-data | ^1.0.4 | ■■■■■■■■ |
```

来源: frontend/package.json:1-33

4. 功能逐点拆解

4.1 应用启动与基础装配

场景

应用启动时需要一次性挂载状态管理、路由系统与 UI 组件库。

需求

保证所有页面共享同一套路由、全局样式和后台登录态。

实现方案

在 `frontend/src/main.ts:12-18` 中创建 Vue 应用实例，并依次注册 Pinia、Router、Element Plus。

选型理由

该实现足够直接，没有引入额外框架包装层，便于维护单仓库博客场景。

核心源码定位

- `frontend/src/main.ts:12-18`

```
const dyxApp = createApp(App);
const dyxPinia = createPinia();

dyxApp.use(dyxPinia);
dyxApp.use(router);
dyxApp.use(ElementPlus);
dyxApp.mount('#app');
```

替代方案对比

- 替代方案：在 `App.vue` 内部再二次注入插件。
- 弃用原因：插件装配属于应用入口职责，放在 `main.ts` 更清晰，也更符合 Vue 社区习惯。

4.2 前后台路由分区与懒加载

场景

站点同时存在公开页面与后台管理页面，需要逻辑隔离但共用同一应用。

需求

通过路由前缀区分权限域，并尽量减少首屏包体积。

实现方案

`frontend/src/router/index.ts:9-77` 定义 `webRoutes` 与 `adminRoutes` 两组路由；各页面组件通过 `() => import(...)` 动态导入，实现路由级懒加载。

选型理由

- 分区后结构直观。
- 动态导入可以降低初始包体积，后台页面不会阻塞前台首屏。

核心源码定位

- frontend/src/router/index.ts:9-27
- frontend/src/router/index.ts:35-68
- frontend/src/router/index.ts:74-77

```
const router = createRouter({
  history: createWebHistory(),
  routes: [...webRoutes, ...adminRoutes]
});
```

替代方案对比

- 替代方案：全部页面静态导入。
- 弃用原因：后台页面较多，静态导入会拉高首屏包体积；当前博客站点更适合按路由切包。

4.3 后台登录态与路由守卫

场景

后台页面必须在登录后访问，未登录时需要跳转到登录页，并保留回跳地址。

需求

- 登录成功后保存 token 和用户信息。
- 未登录访问受保护页面时自动跳转。
- 已登录再次访问登录页时直接进入仪表盘。

实现方案

- frontend/src/stores/auth.ts:26-61 使用 Pinia 保存 token 与用户信息。
- 状态持久化到 sessionStorage，键名为 dyx-admin-token 与 dyx-admin-user。
- frontend/src/router/index.ts:83-97 使用 beforeEach 守卫拦截带 requiresAuth 的后台路由。

选型理由

- Pinia 足够轻量，适合当前只管理认证状态的场景。

- 使用 sessionStorage 而非 localStorage，更偏向“浏览器会话级后台登录”，降低长期残留风险。

核心源码定位

- frontend/src/stores/auth.ts:14-59
- frontend/src/router/index.ts:83-97

```
if (to.meta.requiresAuth && !authStore.isAuthenticated) {  
  return {  
    path: '/dyx-manager/login',  
    query: { redirect: to.fullPath }  
  };  
}
```

替代方案对比

- 替代方案：纯内存态，不持久化。
- 弃用原因：页面刷新后会立刻丢失登录态，后台体验较差。
- 替代方案：持久化到 localStorage。
- 弃用原因：登录有效期更长，但后台管理场景下安全边界更松；当前代码显式清理 localStorage，说明设计偏向会话级存储（frontend/src/stores/auth.ts:45-58）。

4.4 Axios 拦截器与统一错误处理

场景

需要统一处理 API 前缀、后台 Token 注入，以及 401/403 错误反馈。

需求

- 前后台 API 共用 /api 前缀。
- 后台接口自动附带 Bearer Token。
- 登录过期时清空状态并跳转登录页。

实现方案

frontend/src/api/http.ts:8-79 创建 publicHttp 与 adminHttp 两个 Axios 实例：

- 两者共享 baseUrl：'/api' 与超时配置。
- adminHttp 通过请求拦截器自动注入 Authorization。

- 响应拦截器把非 `code === 200` 的统一返回结构转换为 `Promise reject`。
- 遇到 401 时通过 `redirectToLogin()` 清空登录态并跳转。

选型理由

把后台行为封装在独立实例中，避免公开接口被无意义地附加认证逻辑。

核心源码定位

- `frontend/src/api/http.ts:8-12`
 - `frontend/src/api/http.ts:34-47`
 - `frontend/src/api/http.ts:53-59`
 - `frontend/src/api/http.ts:65-77`
- ```
adminHttp.interceptors.request.use((config) => {
 const authStore = useAuthStore();
 if (authStore.token) {
 config.headers.Authorization = `Bearer ${authStore.token}`;
 }
 return config;
});
```

### 替代方案对比

- 替代方案：在每个 API 函数手工拼接 `Authorization` 请求头。
- 弃用原因：重复代码多，且容易漏掉新增接口。

## 4.5 首页聚合展示

### 场景

首页需要一次性展示 Hero 区、最新博客、最新动态、精选项目、最新荣誉与足迹地图。

### 需求

减少首屏接口往返次数，并保持首页数据结构稳定。

### 实现方案

- `frontend/src/views/web/HomeView.vue:251-259` 通过 `getHomeData()` 拉取聚合首页数据。
- `HomeData` 类型在 `frontend/src/api/modules/site.ts:172-180` 定义。

- 页面内部再根据 `heroConfig`、`footprints` 等字段拆解为具体渲染块。

#### 选型理由

首页是典型聚合场景，使用单接口拉取比页面层多次拼装请求更高效，也更容易让前后端围绕同一 DTO 迭代。

#### 核心源码定位

- `frontend/src/api/modules/site.ts:169-180`
- `frontend/src/views/web/HomeView.vue:114-121`
- `frontend/src/views/web/HomeView.vue:241-260`

#### 替代方案对比

- 替代方案：博客、项目、荣誉、足迹分别独立请求。
- 弃用原因：请求数更多，首页加载链路更长，且页面层需要协调多个 `loading/error` 状态。

## 4.6 Hero 配置解析

#### 场景

首页首屏不是固定模板，而是允许后台通过配置调整眉题、标题、副标题、标签和图片，同时需要在不同屏宽下保持稳定的视觉节奏。

#### 需求

- 配置缺失时也能安全回退。
- 需要保持页面结构可预测。
- 首屏左侧文字与右侧主图在桌面端要有明确层级与留白，不因默认字号过小导致视觉重心不足。

#### 实现方案

`frontend/src/api/modules/site.ts:196-255` 提供：

- `createDefaultHeroConfig()`：生成默认配置。
- `parseHeroConfig()`：解析 `profile` 中的 JSON 配置，失败则回退默认值。

`frontend/src/views/web/HomeView.vue` 在渲染层追加了针对 Hero 首屏的响应式样式控制：

- 放大 eyebrow、标题、副标题与按钮尺寸，强化首屏层级。
- 调整 heroGridClass，让右侧图片列在大屏下略大于左侧文案列。
- 提升外层横向 padding、列间距以及文案区右内边距，让左文右图之间的呼吸感更接近设计参考图。
- 增大主图最大高度，并在大屏下增加左侧留白，使图片主体更靠右展示。

#### 选型理由

把配置解析逻辑放在 API/types 层，可以让 View 层专注于渲染，而针对视觉节奏的调整保留在 View 层完成，也更符合首屏样式按页面语义微调的需求。

#### 核心源码定位

- frontend/src/api/modules/site.ts:196-255
- frontend/src/views/web/HomeView.vue:20-101
- frontend/src/views/web/HomeView.vue:444-468

```
const heroGridClass = computed(() =>
 hasHeroImage.value
 ? "lg:grid-cols-[minmax(0,0.92fr)_minmax(0,1.08fr)] xl:grid-cols-[minmax(0,0.9fr)_minmax(0,1.1fr)]"
 : "lg:grid-cols-1"
);
```

#### 替代方案对比

- 替代方案：仅调整标题字号，不改布局间距与图片占比。
- 弃用原因：只能局部放大文本，无法同时改善左侧内容密度、按钮存在感和右侧图片的展示重心，整体观感仍会偏“缩在一起”。

## 4.7 足迹地图渲染

#### 场景

首页第二屏需要用地图展示地理足迹，并兼顾移动端与桌面端交互提示。

#### 需求

- 支持深浅主题。
- 允许高德地图底图与自定义边界/标签叠加。
- 在缩放级别变化时动态控制国家名、城市名、边界显示。

## 实现方案

frontend/src/components/web/HomeFootprintMap.vue:112-359 使用高德地图 SDK 与本地地理数据工具：

- 通过 loadAmapSdk() 按需加载 SDK。
- 使用 featureToPolygonPaths、getMapBoundaryItems、getWorldCountryLabelItems 等工具生成覆盖物。
- 根据 zoom 阈值动态控制国家标签、城市边界、已访问城市标签显隐。

## 选型理由

地图是高度定制化模块，直接基于 AMap + 本地边界数据做二次绘制，比通用图表库更容易实现足迹高亮、标签策略和视觉定制。

## 核心源码定位

- frontend/src/components/web/HomeFootprintMap.vue:113-127
- frontend/src/components/web/HomeFootprintMap.vue:197-205
- frontend/src/components/web/HomeFootprintMap.vue:319-331

```
function ensureWorldLayer(AMap: AMapNamespace): void {
 if (!mapInstance || !AMap.DistrictLayer?.World) {
 return;
 }
 if (!worldLayer) {
 worldLayer = new AMap.DistrictLayer.World({
 zIndex: 1,
 zooms: [MIN_ZOOM, MAX_ZOOM],
 });
 worldLayer.setMap(mapInstance);
 }
 worldLayer.setStyles(getWorldLayerStyles());
}
```

## 替代方案对比

- 替代方案：用静态图片或纯 SVG 地图渲染。
- 弃用原因：交互缩放、拖拽、城市级标注能力较弱，难以达到当前视觉效果。

## 4.9 关于页作品区横向滚动优化

### 场景

ProfileView 中的“个人作品”和“荣誉时间线”都包含横向滚动区域。除了滚轮横向滚动外，桌面端用户还希望能像拖拽内容轨道一样直接拖动区域，并且拖拽时不要触发页面滚动或误点卡片内容。

## 需求

- 作品区与荣誉区在滚轮交互时只滚动各自区域。
- 支持鼠标拖拽横向滚动。
- 拖拽结束后不要误触发卡片点击。
- 荣誉时间线在桌面端滚轮滚动时保持线性跟手，不出现输入结束后的持续自动滚动。
- 荣誉时间线桌面端的滚轮步幅需要比移动端更稳，避免单次滚动过冲。
- 不影响移动端原生触摸横滑。

## 实现方案

在 frontend/src/views/web/ProfileView.vue 中：

- 为作品区与荣誉区滚动容器都绑定 wheel、pointerdown、pointermove、pointerup、pointercancel、pointerleave 和捕获阶段 click 事件。
- 抽出 handleHorizontalWheel()，当鼠标位于横向滚动区域内时始终 preventDefault()，并把滚轮输入直接转换为 scrollLeft 变化，从而阻止整页继续滚动，同时保持“滚多少滑多少”的线性手感。
- 抽出 handleHorizontalPointerDown/Move/Up()，记录起始指针位置与滚动偏移，支持鼠标拖拽横向移动；拖拽过程中显式 preventDefault()，避免文本选择和浏览器默认行为干扰跟手性。
- 使用 shouldSuppressDragClick 在拖拽后短暂抑制点击，避免把“拖拽结束”识别成打开素材或跳转链接。
- 对荣誉时间线桌面端单独做步幅归一化处理：通过给通用滚轮逻辑传入缩放系数来减小桌面端单次滚动距离，而不是重新构造事件对象；手机端仍保持原始映射。
- 荣誉卡片内的图片 / 视频媒体区域通过 @pointerdown.stop 与横向拖拽容器隔离，避免点击预览图片或操作视频控件时被误判为横向拖拽。
- 为 .works-scroll / .honors-scroll 增加 cursor: grab / grabbing、user-select: none、-webkit-overflow-scrolling: touch 与 overscroll-behavior-x: contain，优化可拖拽反馈和横向滚动体验。

## 选型理由

该方案只增强横向滚动容器本身的交互，不改作品卡片与荣誉时间线的数据结构和内容布局，改动范围小，但能明显提升桌面端浏览手感。

#### 核心源码定位

- frontend/src/views/web/ProfileView.vue:68-78
- frontend/src/views/web/ProfileView.vue:265-273
- frontend/src/views/web/ProfileView.vue:568-650

#### 替代方案对比

- 替代方案：引入第三方拖拽/轮播库统一处理横向滑动。
- 弃用原因：当前需求只涉及两个简单横向滚动容器，使用原生事件即可完成，没必要额外增加依赖和复杂度。

## 4.10 WebLayout 主题与导航可见性控制

#### 场景

公开站点既要支持深浅主题切换，又要在首页全屏滚动场景中控制顶部导航显隐。

#### 需求

- 支持自动主题与手动主题切换。
- 主题选择应具备持久化能力。
- 首页与普通内容页的布局行为不同。

#### 实现方案

frontend/src/layouts/WebLayout.vue:17-145 负责：

- 使用 provide() 向子组件暴露主题与导航可见性控制函数。
- 通过 localStorage 读取/保存用户手动主题选择。
- 在没有手动主题时，按 8:00~18:00 使用浅色、其余时间使用深色。
- 根据当前路由是否为首页，切换全屏滚动布局与普通文档布局。

#### 选型理由

把主题和顶栏显隐集中在

WebLayout，比散落在每个页面中更容易保持全站一致性，也避免多页面重复实现主题切换。

核心源码定位

- frontend/src/layouts/WebLayout.vue:17-33
- frontend/src/layouts/WebLayout.vue:35-49
- frontend/src/layouts/WebLayout.vue:64-118
- frontend/src/layouts/WebLayout.vue:121-145

```
const layoutClass = computed(() =>
 isHomeRoute.value ? 'h-screen overflow-hidden' : 'min-h-screen overflow-x-hidden'
);

function toggleTheme(): void {
 const nextTheme: ThemeMode = theme.value === 'dark' ? 'light' : 'dark';
 syncDocumentTheme(nextTheme);
 window.localStorage.setItem(THEME_STORAGE_KEY, nextTheme);
 clearScheduledThemeTimer();
}
```

替代方案对比

- 替代方案：每个页面自己维护主题状态。
- 弃用原因：容易出现主题不一致和重复逻辑，尤其是首页与普通页布局差异较大。

## 4.9 地图 SDK 延迟加载与地理数据缓存

场景

足迹地图属于重模块，不适合阻塞首页首屏；同时地理边界数据解码成本较高，需要避免重复计算。

需求

- 高德地图脚本只在真正需要时加载。
- 地理边界数据只解码一次，多次缩放和切换时复用结果。
- 缺失 API Key 时应明确失败，而不是静默异常。

实现方案

- frontend/src/utis/amapLoader.ts:24-87 使用 Promise 缓存 SDK 加载过程，避免重复插入脚本。



- frontend/src/utils/footprintGeo.ts:166-181 对中国、省级、世界 GeoJSON 做规范化与缓存。
- boundaryCache 与 normalizedProvinceCache 用于复用已经处理过的边界数据。

#### 选型理由

这类地图模块的性能瓶颈不在普通页面渲染，而在 SDK 下载与地理数据解码。当前做法把两者都延迟并缓存，是非常符合场景的优化。

#### 核心源码定位

- frontend/src/utils/amapLoader.ts:24-87
  - frontend/src/utils/footprintGeo.ts:140-181
- ```
if (amapLoaderPromise) {  
  return amapLoaderPromise;  
}  
  
const normalizedProvinceCache = new Map<string, GeoJsonCollection>();  
const boundaryCache = new Map<string, GeoJsonFeature[]>();
```

替代方案对比

- 替代方案：应用启动时就加载地图 SDK。
- 弃用原因：会拖慢不访问首页足迹模块的用户首屏。
- 替代方案：每次进入地图都重新解析 GeoJSON。
- 弃用原因：重复 CPU 开销大，缩放和切页体验会变差。

4.10 仪表盘图表与异步组件拆分

场景

后台仪表盘需要展示摘要卡片、趋势图和热门页面排行，但图表库本身较重。

需求

- 首屏先渲染摘要信息。
- 图表模块按需加载，避免占用后台所有页面的初始包体积。
- 统计卡片需要从同一份摘要数据派生。

实现方案

frontend/src/views/admin/AdminDashboardView.vue:75-215 采用:

- defineAsyncComponent() 延迟加载 AdminDashboardCharts.vue。
- summary 作为单一数据源, 再派生 trendPoints、deviceStats、topPages、summaryCards。
- onMounted() 调用 getDashboardSummary() 拉取摘要数据。

选型理由

仪表盘是后台中的“重量页面”, 但图表能力又只在该页面需要。异步拆分组件比把图表逻辑混进全局入口更节省资源。

核心源码定位

- frontend/src/views/admin/AdminDashboardView.vue:76-95
- frontend/src/views/admin/AdminDashboardView.vue:97-172
- frontend/src/views/admin/AdminDashboardView.vue:201-215

```
const AdminDashboardCharts = defineAsyncComponent(
  () => import('@/components/admin/AdminDashboardCharts.vue')
);

onMounted(() => {
  void loadDashboardSummary();
});
```

替代方案对比

- 替代方案: 图表组件静态导入。
- 弃用原因: 所有后台页面都会为仪表盘图表付出初始加载成本。

4.11 后台导航与管理页组织

场景

后台功能较多, 需要稳定导航结构, 让内容管理与系统管理分区清晰。

需求

支持桌面端侧栏与移动端抽屉导航, 并保持路由驱动菜单高亮。

实现方案

frontend/src/layouts/AdminLayout.vue:136-182 用数组声明导航组，再交给 Element Plus 菜单组件渲染。

当前后台按“概览 / 首页管理 / 内容管理 / 个人资料 / 系统”分组，首页管理下已包含首屏管理、足迹管理、更新节奏等模块。

选型理由

数据驱动菜单比手写大量模板更易维护，新增页面时只改一处即可。

核心源码定位

- frontend/src/layouts/AdminLayout.vue:136-177
- frontend/src/layouts/AdminLayout.vue:179-182

替代方案对比

- 替代方案：在模板中逐个写死菜单项。
- 弃用原因：层级多、重复代码多，不利于维护后台导航分组。

4.12 后台媒体选择与业务裁剪协作

场景

后台多个业务表单都需要复用媒体库，但头像、首页首屏背景图、首页右侧人物图又需要不同的裁剪预览与导出结果。

需求

- 媒体库负责统一上传、列表选择与资源复用。
- 业务表单可以直接使用原图，也可以按业务场景单独裁剪。
- 裁剪预览需要尽量贴近最终展示效果。
- 后台已选图片与媒体库图片在桌面端需要支持点击放大预览，便于编辑时核对细节。

实现方案

- frontend/src/views/admin/AdminMediaPicker.vue:69-208
提供媒体弹窗、上传入口、资源卡片与已选状态。

- 图片资源在弹窗中拆分为“直接使用”和“裁剪后使用”两条路径，避免把媒体库强耦合成裁剪器。
- frontend/src/components/admin/BusinessImageCropper.vue:1-195 根据 avatar、hero-background、hero-portrait 三种业务模式切换预览布局。
- 当前裁剪框已允许自由缩放，预览区根据实时裁剪结果自适应，不再强制固定比例缩放框。
- AdminMediaPicker 内的已选图片列表与媒体库图片卡片都改为 el-image，并启用 preview-src-list + preview-teleported，使后台编辑场景下可直接点击缩略图查看大图。

选型理由

把“媒体资源管理”和“业务图片加工”拆开后，媒体库仍然保持通用，而具体业务页面可以决定是否需要裁剪以及采用哪种预览样式；同时复用 Element Plus 原生预览能力，比额外维护一套自定义图片灯箱更轻、更稳定。

核心源码定位

- frontend/src/views/admin/AdminMediaPicker.vue:4-45
- frontend/src/views/admin/AdminMediaPicker.vue:104-147
- frontend/src/views/admin/AdminMediaPicker.vue:349-354
- frontend/src/components/admin/BusinessImageCropper.vue:19-46
- frontend/src/components/admin/BusinessImageCropper.vue:72-169
- frontend/src/components/admin/BusinessImageCropper.vue:256-308

替代方案对比

- 替代方案：继续使用普通 `<img>` 作为后台缩略图。
- 弃用原因：编辑时无法直接核对原图细节，尤其在荣誉、头像、首屏配图等场景下不利于内容确认。
- 替代方案：为后台单独实现自定义灯箱组件。
- 弃用原因：当前只需要基础放大预览，复用 Element Plus 现有能力即可，没必要额外增加维护成本。

5. 数据流与状态管理

5.1 组件生命周期

首页页面在挂载时会获取聚合数据、组装足迹视图模型，再根据滚动位置控制不同 section。HomeView.vue 还负责把足迹相关数据映射为地图可消费结构。

后台仪表盘则在 AdminDashboardView.vue:201-215 挂载时加载摘要数据，再通过多个 computed 派生卡片、趋势与排行视图。

5.2 全局状态管理

5.1 组件生命周期

首页页面在挂载时会获取聚合数据、组装足迹视图模型，再根据滚动位置控制不同 section。HomeView.vue 还负责把足迹相关数据映射为地图可消费结构。

5.2 全局状态管理

当前全局状态集中在 frontend/src/stores/auth.ts:9-77，范围较克制：

- 只保存后台认证信息。
- 业务数据（博客、动态、留言等）按页面请求，不做复杂全局缓存。
- 登录态默认写入 sessionStorage，同时显式清理同名 localStorage 键，避免旧实现残留。
- 后台登录用户 ID 当前按 string | number 处理，用于规避 Long 主键在浏览器中的精度丢失问题。

这说明前端设计偏向“页面拉取 + 局部状态”，而不是 Redux/Vuex 风格的大型全局仓库。

5.3 路由守卫

frontend/src/router/index.ts:83-97 使用前置守卫完成：

- 未登录拦截后台访问。
- 登录后拦截重复进入登录页。
- 通过 query 保存重定向目标。

5.4 缓存策略

前端未发现复杂客户端缓存层：

- 登录态缓存：sessionStorage

- 业务数据缓存：主要依赖后端 @Cacheable，前端未见 SW/IndexedDB/Query 缓存框架。

6. 安全与性能

6.1 性能

资源拆包

frontend/vite.config.ts:4-33 通过 manualChunks 手工分包：

- vendor-core: Vue / Router / Pinia
- vendor-ui: Element Plus
- vendor-charts: ECharts
- vendor-map: 地图数据与足迹工具

这是一个非常明确的性能优化信号，说明项目已针对地图、图表等重依赖做包体切分。

路由懒加载

路由组件全部采用动态导入，见 frontend/src/router/index.ts:15-26 与 frontend/src/router/index.ts:37-66。

6.2 安全

Token 注入

后台接口统一从 Pinia 读取 token 并注入 Authorization，见 frontend/src/api/http.ts:53-59。

登录失效处理

401 时会清理认证状态并强制跳转登录页，避免前端继续带失效 token 工作，见 frontend/src/api/http.ts:65-77。

Token 刷新机制

当前未发现 Refresh Token 或无感刷新实现，文档中应明确说明“未实现”。

6.3 SSR/CSR 选型

当前为典型 CSR 架构：

- 入口为 index.html
- 使用 createWebHistory()
- 由 Nginx try_files 支持前端路由刷新

未发现 SSR/Nuxt 相关配置，因此该项目明确是 CSR。

7. 测试与质量保障

7.1 测试覆盖现状

当前仓库未发现以下内容：

- 前端单元测试文件
- E2E 测试目录
- Vitest / Jest / Cypress / Playwright 配置

因此该部分在最终交付中应写为：****当前仓库未纳入前端自动化测试基建****。

7.2 自动化流水线

当前仓库未发现：

- .github/workflows/
- GitLab CI 配置
- Jenkinsfile

因此不能将前端 CI/CD 写成已具备能力。

7.3 代码规范

当前仓库未发现 ESLint、Prettier 配置文件；可观察到代码整体保持 TypeScript + Vue SFC 结构化写法，但缺少可执行的静态检查配置。

8. 部署与运维

8.1 构建

前端构建命令来自 frontend/package.json:5-8:

```
"build": "vue-tsc --noEmit && vite build"
```

8.2 Nginx 托管与 API 反向代理

docs/nginx.conf:4-42 说明了生产部署方式:

- / 与 /dyx-manager/ 指向前端静态资源目录。
- /api/dyx-manager/ 与 /api/ 反向代理到 localhost:8080。
- try_files \$uri \$uri/ /index.html; 负责支持 Vue Router 刷新。

8.3 环境变量

前端存在:

- frontend/.env.local
- frontend/.env.production

但本次文档编写基于源码与公开配置结构, 不对未读取内容做臆测说明。

9. 现状结论与补充建议

9.1 已具备能力

- 前后台一体化 SPA 架构
- 路由分区与后台鉴权守卫
- Pinia 登录态管理
- Axios 拦截器与统一失败跳转
- Vite 手工分包
- 首页聚合接口消费
- 高定制足迹地图渲染
- Nginx 反向代理部署方案

9.2 当前未发现或未实现能力

- SSR
- Token 自动刷新
- 前端单测 / E2E
- ESLint / Prettier
- CI/CD 流水线
- 文档 lint 配置

9.3 读者复现建议

若读者要在 10 分钟内定位源码，建议优先按以下顺序阅读：

- `frontend/src/main.ts`
- `frontend/src/router/index.ts`
- `frontend/src/stores/auth.ts`
- `frontend/src/api/http.ts`
- `frontend/src/api/modules/site.ts`
- `frontend/src/views/web/HomeView.vue`
- `frontend/src/components/web/HomeFootprintMap.vue`
- `frontend/src/layouts/AdminLayout.vue`